

Informe de Profundización sobre las Vulnerabilidades OWASP Top 10 (Edición 2021)

1. Resumen Ejecutivo

El presente informe técnico ofrece un análisis exhaustivo de las diez vulnerabilidades de seguridad más críticas para las aplicaciones web, según el marco de referencia **OWASP Top 10: 2021**.¹ Este documento, reconocido a nivel mundial por desarrolladores y profesionales de la ciberseguridad, no es una lista estática, sino un barómetro en constante evolución que refleja los riesgos más apremiantes del panorama digital actual.² La edición de 2021 marca un punto de inflexión estratégico al alejarse de la mera catalogación de síntomas de seguridad para centrarse en las causas fundamentales de los fallos, promoviendo una visión de seguridad más proactiva y arraigada en las etapas iniciales del desarrollo de software.¹

El análisis detallado de cada vulnerabilidad —desde el control de acceso deficiente (A01) hasta la falsificación de solicitudes del lado del servidor (A10)— revela que los atacantes modernos rara vez explotan un solo fallo. En su lugar, construyen cadenas de ataque complejas que encadenan múltiples vulnerabilidades para lograr objetivos más destructivos, como la exfiltración masiva de datos o el compromiso total del sistema. Por lo tanto, el enfoque de este informe no solo se limita a la descripción y mitigación individual de cada riesgo, sino que también subraya la imperativa necesidad de una estrategia de seguridad holística. Esta estrategia debe integrar la protección en cada etapa del ciclo de vida del desarrollo de software (SDLC) y adoptar principios de diseño seguro, como el modelo de confianza cero y la defensa en profundidad, para construir aplicaciones que sean inherentemente resistentes a los ataques.

2. Introducción al OWASP Top 10: Un Marco de Concienciación Crítico y su Evolución

El Open Web Application Security Project (OWASP) es una fundación sin fines de lucro que trabaja para mejorar la seguridad del software.¹ Su documento insignia, el OWASP Top 10, es una lista de consenso global entre expertos que clasifica los riesgos de seguridad más críticos para las aplicaciones web.² Desde su primera publicación en 2003, la lista ha servido como un recurso fundamental para la concienciación y como un punto de partida para que las organizaciones produzcan código más seguro.¹ El fracaso de una organización en abordar estos riesgos a menudo es interpretado por los auditores como una indicación de deficiencias en otros estándares de cumplimiento.²

La edición de 2021 representa una de las actualizaciones más significativas, introduciendo un cambio de paradigma notable. En lugar de enfocarse únicamente en las manifestaciones o síntomas de las fallas de seguridad, la lista se reestructura para abordar las causas raíz. Esto se alinea con la creciente adopción de la práctica de "mover a la izquierda" (*shift left*) en la industria, que implica integrar la seguridad lo más temprano posible en el ciclo de desarrollo.³ Esta transición se evidencia en varios cambios importantes:

- **Nuevas Categorías:** Se agregaron tres categorías completamente nuevas que reflejan la evolución de las amenazas. A04:2021 - Diseño Inseguro, A08:2021 - Fallos de Integridad de Software y Datos y A09:2021 - Fallos de Registro y Monitoreo de Seguridad abordan áreas que a menudo se pasaban por alto, como la arquitectura del software, la cadena de suministro de componentes y la visibilidad de los incidentes.¹
- **Cambios de Nomenclatura y Alcance:** Se redefinieron cuatro categorías para reflejar una comprensión más profunda de los fallos. Por ejemplo, la vulnerabilidad A02, antes conocida como "Exposición de Datos Sensibles", se renombró a Fallos Criptográficos.¹ Esta es una distinción crucial: la exposición de datos es el *resultado* o síntoma de una mala práctica, mientras que la criptografía deficiente es la *causa raíz* que permite que el síntoma se manifieste. Del mismo modo, A07 - Fallos de Identificación y Autenticación se expandió para incluir fallos de identificación, más allá de la simple "autenticación rota".⁵
- **Consolidación:** La lista de 2021 también consolidó algunas categorías para simplificar el marco. El *Cross-Site Scripting* (XSS), que era una categoría independiente en 2017, ahora se agrupa bajo A03 - Inyección, reconociéndolo como una forma de ataque de inyección de código del lado del cliente.¹

Este movimiento hacia la causa fundamental, en lugar del síntoma, representa un cambio en el consenso de la comunidad de seguridad. Un fallo de implementación de codificación, como el *buffer over-read* en Heartbleed, es a menudo la manifestación de un error de diseño subyacente que no consideró la validación de entrada como una necesidad arquitectónica

fundamental.⁹ Esta distinción subraya que una implementación perfecta no puede solucionar un diseño inherentemente defectuoso.⁴

3. Análisis de las Vulnerabilidades: Un Deep Dive Técnico

3.1. A01:2021 - Control de Acceso Roto

Esta vulnerabilidad subió del puesto 5 al 1 en 2021, lo que la convierte en el riesgo de aplicación web más crítico.² Se probó la existencia de algún tipo de fallo de control de acceso en el 94% de las aplicaciones analizadas.³ Esta vulnerabilidad se produce cuando no se aplican correctamente los controles de acceso y la autorización, permitiendo a un atacante acceder a funcionalidades o datos para los que no tiene permisos.¹⁰

Ejemplos de Explotación

- **Referencias Directas Inseguras a Objetos (IDOR):** Es una de las formas más comunes de ataque. La aplicación utiliza una referencia directa a un objeto interno, como un ID de pedido o de documento (<https://example.com/orders?id=1234>).¹² Un atacante puede manipular este parámetro (id=1235) para acceder a datos o recursos de otro usuario sin una validación de autorización adecuada.¹² Esto puede resultar en la exposición de datos sensibles, como información personal y financiera.¹²
- **Escalada de Privilegios:** Los atacantes pueden ascender de un usuario de bajo privilegio a un administrador al manipular parámetros de la URL, *headers* o *cookies*, o al eludir los flujos de trabajo de varios pasos que carecen de controles de acceso consistentes en todas sus etapas.¹³ Por ejemplo, un usuario estándar de un sistema ERP puede acceder a un módulo de gestión financiera simplemente alterando la URL debido a la falta de validación de roles en el *backend*.¹³
- **Exposición de APIs:** Las interfaces de programación de aplicaciones (APIs) mal configuradas, que no validan los permisos de los usuarios, pueden permitir el acceso no autorizado a bases de datos completas o a funciones restringidas.¹³ Un atacante puede manipular el ID de usuario en una solicitud a la API para acceder a los pedidos de otros clientes, lo que podría llevar al robo de identidad y fraude.¹³

Mitigación

La prevención de fallos de control de acceso debe implementarse principalmente en el lado del servidor, ya que los atacantes no pueden modificar la lógica de verificación allí.¹⁵

- **Principio de Mínimo Privilegio (PoLP):** Este es el fundamento de una defensa robusta.¹⁴ Se deben conceder a los usuarios y sistemas solo los permisos estrictamente necesarios para realizar sus funciones.¹⁶
- **Control de Acceso Basado en Roles (RBAC):** Implementar mecanismos de control de acceso basados en roles que apliquen la propiedad de los registros y los límites de negocio únicos. Esto asegura que solo los usuarios con los roles definidos puedan acceder a ciertas funciones.¹²
- **Validación del Lado del Servidor y Negación por Defecto:** Se debe negar el acceso a todos los recursos excepto a aquellos que son explícitamente públicos.¹⁵ Todas las comprobaciones de autorización deben realizarse en código confiable del lado del servidor.¹⁴
- **Gestión Segura de Sesiones:** Los tokens de sesión y de autorización (como los JSON Web Tokens o JWT) deben ser de corta duración para minimizar la ventana de oportunidad para un atacante.¹⁵

3.2. A02:2021 - Fallos Criptográficos

Esta vulnerabilidad, antes conocida como "Exposición de Datos Sensibles", se centra en la protección inadecuada de la información durante su tránsito y en reposo.¹ Los fallos en la criptografía pueden llevar a la divulgación de datos confidenciales, el acceso no autorizado a información sensible (como registros financieros y propiedad intelectual) y el incumplimiento de normativas como el GDPR y el PCI DSS.⁷ Las causas pueden variar desde el uso de algoritmos débiles hasta una gestión de claves deficiente.¹

Ejemplos de Explotación

- **MITM (Man-in-the-Middle) por Degradación de Conexión:** Un sitio que no fuerza el uso de TLS para todas las páginas permite que un atacante degrade la conexión de HTTPS a HTTP en una red insegura. Esto le permite interceptar las solicitudes, robar la *cookie* de sesión del usuario y secuestrar su sesión autenticada.⁷
- **Divulgación de Datos en Texto Claro:** Un fallo de inyección SQL en una aplicación que utiliza cifrado automático de la base de datos puede permitir que un atacante recupere números de tarjetas de crédito en texto claro, ya que los datos se descifran automáticamente al ser solicitados.⁷
- **Contraseñas Almacenadas de forma Insegura:** Si la base de datos de contraseñas

utiliza hashes simples o sin *salt*, un atacante puede recuperar el archivo de la base de datos y usar tablas *rainbow* o ataques de fuerza bruta basados en GPU para descifrar rápidamente todas las contraseñas.⁷

Mitigación

- **Clasificación de Datos:** El primer paso es identificar y clasificar qué datos son sensibles según las leyes de privacidad o las necesidades del negocio.⁷ Se recomienda evitar almacenar datos sensibles innecesariamente.⁷
- **Cifrado Robusto:** Se deben utilizar algoritmos fuertes y actualizados (como AES o RSA) con una gestión de claves adecuada y segura.⁷ Todos los datos sensibles deben cifrarse en reposo.⁷
- **Transmisión Segura:** El tráfico de datos debe cifrarse en tránsito utilizando protocolos seguros como TLS con cifradores de confidencialidad anticipada. Se deben deshabilitar los protocolos heredados como FTP y SMTP para el transporte de datos confidenciales.⁷
- **Almacenamiento de Contraseñas:** Las contraseñas deben almacenarse utilizando funciones de *hashing* adaptativas y con *salt* (PBKDF2, bcrypt, scrypt o Argon2) para resistir los ataques de fuerza bruta.⁷

3.3. A03:2021 - Inyección

La inyección sigue siendo una de las vulnerabilidades más críticas y generalizadas, presente en el 94% de las aplicaciones y con el segundo mayor número de ocurrencias en los datos de OWASP.⁵ Ocurre cuando un atacante inserta datos maliciosos no validados, filtrados o saneados en un comando o consulta, lo que puede resultar en la ejecución de código no intencionado.¹⁰ Esta categoría ahora incluye el *Cross-Site Scripting* (XSS).¹

Ejemplos de Explotación

- **Inyección SQL:** Un atacante inserta una cadena de SQL maliciosa en un campo de entrada (' OR 1=1 --) que modifica la lógica de una consulta original.²³ Esto le permite eludir los controles de autenticación, acceder a datos no autorizados, o incluso manipular o eliminar información de la base de datos.²¹
- **Inyección de Comandos del Sistema Operativo:** Al manipular la entrada de un formulario, un atacante puede engañar a la aplicación para que ejecute comandos del sistema operativo en el servidor subyacente.²¹ Esto puede llevar al compromiso total del sistema y a la ejecución de código arbitrario.²⁴
- **Cross-Site Scripting (XSS):** Un atacante inyecta scripts maliciosos del lado del cliente (como JavaScript) en una página web. Cuando otro usuario visita la página, su navegador ejecuta el script, lo que puede llevar al robo de credenciales de sesión, el registro de

pulsaciones de teclas o el redireccionamiento a sitios maliciosos.⁸

Mitigación

La principal defensa contra las inyecciones es la separación estricta entre los datos y los comandos.

- **Consultas Parametrizadas y Procedimientos Almacenados:** Esta es la estrategia más eficaz contra la inyección SQL.¹ Al usar parámetros, el intérprete de la base de datos trata la entrada del usuario como un valor de datos, no como parte del comando SQL, lo que evita que se ejecute código malicioso.
- **Validación de Entradas:** Se debe validar toda la entrada del usuario antes de procesarla, preferiblemente utilizando listas blancas para aceptar solo datos válidos.²¹
- **Codificación de Salida:** Para prevenir XSS, los datos generados por el usuario que se muestran en la página deben codificarse para que los navegadores los interpreten como texto, no como código ejecutable.⁸

Tipo de Inyección	Objetivo del Ataque	Ejemplo de <i>Payload</i>	Mitigación Principal
SQL Injection	Base de datos	SELECT * FROM users WHERE id = '1' OR '1'='1'--	Consultas parametrizadas, procedimientos almacenados
Command Injection	Servidor web	; cat /etc/passwd	No ejecutar comandos del sistema operativo con entrada del usuario
Cross-Site Scripting (XSS)	Otros usuarios	<script>alert('XSS');</script>	Codificación de salida, saneamiento de datos

3.4. A04:2021 - Diseño Inseguro

Esta es una categoría completamente nueva en 2021, que subraya la importancia de considerar la seguridad desde las etapas de diseño del software, no solo durante la codificación.¹ El diseño inseguro se refiere a las fallas arquitectónicas y a los controles de

seguridad ausentes o ineficaces que, por definición, no pueden ser solucionados con una implementación perfecta.⁴

Ejemplos de Explotación

- **Flujos de Trabajo Inseguros:** Un sistema de recuperación de contraseñas que utiliza preguntas de seguridad (como "¿en qué calle creció?") es inseguro por diseño.²⁷ La vulnerabilidad no está en la implementación, sino en el concepto mismo, ya que las respuestas pueden ser conocidas por terceros, lo que socava la prueba de identidad.²⁸
- **Ausencia de Límites de Velocidad (*Rate Limiting*):** Un API de inicio de sesión que no tiene un límite en la cantidad de intentos fallidos por minuto es vulnerable a ataques de fuerza bruta.²⁹ Esta es una deficiencia de diseño, ya que la lógica de negocio no incluye el control de seguridad necesario.
- **Fallas en la Lógica de Negocio:** Un sistema de comercio electrónico diseñado sin mecanismos de protección contra bots es vulnerable a que los revendedores compren rápidamente grandes cantidades de productos, lo que daña la reputación y la relación con los clientes.²⁸

Mitigación

- **Modelado de Amenazas:** Esta es la principal herramienta para mitigar el diseño inseguro. El modelado de amenazas identifica posibles fallos y amenazas de seguridad en la fase de diseño, mucho antes de que se escriba el código.³
- **Patrones de Diseño Seguro:** Utilizar patrones de diseño de referencia probados para la seguridad.¹ Un ejemplo simple es utilizar el *hashing* para contraseñas en lugar de compararlas en texto plano, lo que es un patrón de diseño seguro.²⁹
- **Integración de la Seguridad en el SDLC:** Los profesionales de seguridad deben participar activamente en la definición de requisitos y en la arquitectura de la aplicación.³¹ Se deben incluir verificaciones de plausibilidad en cada nivel de la aplicación.²⁸
- **Límites de Recursos:** Diseñar los sistemas para limitar el consumo de recursos por usuario o servicio para prevenir ataques de denegación de servicio.²⁸

3.5. A05:2021 - Mala Configuración de Seguridad

La mala configuración de seguridad ocurre cuando la aplicación o el sistema no se configuran correctamente, lo que los deja vulnerables a los ataques.³² Esta vulnerabilidad es a menudo el resultado del uso de configuraciones predeterminadas, la instalación de funciones

innecesarias o la falta de un proceso de endurecimiento de seguridad.³³

Ejemplos de Explotación

- **Credenciales Predeterminadas:** Muchas aplicaciones y sistemas vienen con contraseñas predeterminadas (admin/admin, root/root) que, si no se cambian, permiten a los atacantes obtener acceso no autorizado.³²
- **Exposición de Directorios:** Si la opción de Directory Listing no está deshabilitada en el servidor web, un atacante puede navegar por la estructura de archivos y descargar archivos sensibles, como código fuente o copias de seguridad.³³
- **Configuraciones de la Nube:** Las brechas de datos masivas, como la de Exactis, han ocurrido debido a *buckets* de almacenamiento en la nube (como Amazon S3) con permisos de acceso público configurados incorrectamente, lo que expone millones de registros a Internet.³⁴
- **Mensajes de Error Informativos:** Los mensajes de error detallados (como los rastros de pila) pueden exponer información sensible sobre el sistema, como las versiones de los componentes y las rutas de los archivos, lo que puede ayudar a un atacante a planificar su ataque.³³

Mitigación

- **Proceso de Endurecimiento Repetible:** Se debe implementar un proceso automatizado y repetible para endurecer la configuración de los sistemas. Los entornos de desarrollo, control de calidad y producción deben tener configuraciones idénticas, pero con credenciales distintas.³³
- **Principio de Mínima Superficie de Ataque:** Deshabilitar todas las funciones, servicios, puertos y componentes que no sean estrictamente necesarios para el funcionamiento de la aplicación.³²
- **Gestión de Parches:** Revisar y aplicar de manera oportuna las actualizaciones y parches de seguridad en todos los entornos.³³
- **Auditorías y Pruebas de Penetración:** Las auditorías de seguridad y las pruebas de penetración regulares son esenciales para identificar las configuraciones incorrectas antes de que sean explotadas por atacantes.³²

3.6. A06:2021 - Componentes Vulnerables y Desactualizados

Esta vulnerabilidad, que subió del puesto 9 al 6 en la lista de 2021³, se refiere al uso de bibliotecas,

frameworks o dependencias de terceros con vulnerabilidades de seguridad conocidas.¹ Las

fallas en los componentes pueden tener un impacto severo, ya que a menudo se ejecutan con los mismos privilegios que la aplicación.³⁷

Ejemplos de Explotación

- **Ejecución de Código Remoto (RCE):** El caso de la brecha de Equifax en 2017 es un ejemplo clásico. Los atacantes explotaron una vulnerabilidad de RCE en el *framework* de código abierto Apache Struts que la empresa no había parcheado a tiempo.²¹ Esto les permitió acceder y robar datos de más de 143 millones de personas.²¹
- **Dispositivos IoT sin Parches:** Los dispositivos de Internet de las Cosas (IoT) a menudo son difíciles o imposibles de parchear, lo que los deja permanentemente vulnerables a fallos antiguos. El motor de búsqueda Shodan puede ser utilizado para encontrar sistemas sin parches que aún son susceptibles a vulnerabilidades conocidas como el fallo Heartbleed de 2014.³⁷

Mitigación

La mitigación de esta vulnerabilidad requiere un enfoque proactivo y continuo de gestión de parches.

- **Inventario y Monitoreo Continuo:** Mantener un inventario completo de todos los componentes, tanto del lado del cliente como del servidor, y sus dependencias. Monitorear continuamente fuentes como la Base de Datos Nacional de Vulnerabilidades (NVD) y las Common Vulnerabilities and Exposures (CVE) para nuevas vulnerabilidades.³⁷
- **Análisis de Composición de Software (SCA):** Utilizar herramientas de SCA, como OWASP Dependency Check, para automatizar el proceso de escaneo e identificación de componentes con vulnerabilidades conocidas.³⁷
- **Gestión de Parches:** Implementar una política rigurosa para la aplicación oportuna de parches y la eliminación de componentes y dependencias no utilizados.³⁷ Solo se deben adquirir componentes de fuentes oficiales y seguras.³⁷

Componente	Versión	Vulnerabilidad (CVE)	Fecha de Parche Disponible	Estado de Aplicación
Apache Struts	2.3.34	CVE-2017-5638	2017-03-06	Pendiente
OpenSSL	1.0.1f	CVE-2014-0160	2014-04-07	Aplicado
Librería XYZ	1.2.0	No conocida	N/A	En monitoreo

Nginx	1.21.3	CVE-2022-XXX XX	2022-01-10	En progreso
-------	--------	--------------------	------------	-------------

3.7. A07:2021 - Fallos de Identificación y Autenticación

Anteriormente "Autenticación Rota", esta categoría aborda las debilidades en los sistemas de autenticación y gestión de sesiones.⁵ Permite a los atacantes hacerse pasar por otros usuarios, lo que les da acceso a datos y funciones protegidas.¹⁰

Ejemplos de Explotación

- **Relleno de Credenciales (*Credential Stuffing*):** Los atacantes utilizan listas de pares de nombre de usuario y contraseña filtrados de otras brechas para realizar ataques automatizados de inicio de sesión.⁴⁰ Las aplicaciones que carecen de protección contra este tipo de ataques pueden ser utilizadas como "oráculos" para verificar qué credenciales son válidas.⁴⁰
- **Secuestro de Sesión:** Si la aplicación expone el identificador de sesión en la URL o no lo invalida correctamente al cerrar la sesión o después de un período de inactividad, un atacante puede robar la sesión de un usuario y realizar acciones en su nombre.⁴⁰
- **Prácticas de Contraseñas Débiles:** A pesar de los requisitos de complejidad y rotación, las contraseñas débiles siguen siendo un problema.⁴⁰ El uso de contraseñas predeterminadas o conocidas facilita la entrada de atacantes.⁴¹

Mitigación

- **Autenticación Multifactor (MFA):** La MFA es la defensa más efectiva contra los ataques de relleno de credenciales y fuerza bruta, ya que requiere un segundo factor que es difícil de robar.¹⁴
- **Políticas de Contraseñas Modernas:** Se recomienda seguir las directrices del NIST 800-63b, que favorecen la longitud de la contraseña sobre la complejidad y la rotación forzada.⁴⁰
- **Gestión Segura de Sesiones:** Los IDs de sesión deben generarse con alta entropía, almacenarse de forma segura (no en la URL) y ser invalidados inmediatamente al cerrar la sesión o después de un tiempo de inactividad.¹⁵
- **Límites de Intentos:** Se debe limitar y retrasar progresivamente los intentos de inicio de sesión fallidos para frustrar los ataques automatizados.⁴¹

3.8. A08:2021 - Fallos de Integridad de Software y Datos

Esta nueva categoría se enfoca en las fallas que ocurren cuando el código y la infraestructura no protegen contra las violaciones de integridad.⁴ Se basa en la noción de que confiar ciegamente en actualizaciones de software, bibliotecas o datos de fuentes no verificadas puede llevar a un compromiso grave.

Ejemplos de Explotación

- **Ataques a la Cadena de Suministro de Software:** El ataque a SolarWinds es un ejemplo de libro de texto.⁴³ Los atacantes comprometieron la infraestructura de compilación y actualización de SolarWinds, inyectando código malicioso en una actualización de software legítima.⁴² Esta actualización fue distribuida a más de 18,000 organizaciones, lo que les dio acceso no autorizado a los sistemas de miles de clientes.⁴⁴
- **Deserialización Insegura:** Si una aplicación deserializa datos de una fuente no confiable, un atacante puede manipular los datos serializados para inyectar objetos maliciosos, lo que puede llevar a la ejecución de código remoto en el servidor.⁴²
- **Actualizaciones de Firmware no Firmadas:** Muchos dispositivos, como los *routers* domésticos, no verifican la integridad o el origen de las actualizaciones de firmware mediante firmas digitales.⁴⁴ Esto permite a un atacante distribuir su propio firmware malicioso, comprometiendo el dispositivo sin que el usuario lo sepa.⁴²

Mitigación

- **Firmas Digitales:** Se deben utilizar firmas digitales o mecanismos similares para verificar que el software, las bibliotecas y los datos provienen de una fuente de confianza y no han sido alterados.⁴²
- **Repositorios Verificados:** Se deben obtener las dependencias y bibliotecas de repositorios confiables y, para entornos de alto riesgo, se recomienda el uso de repositorios internos y auditados.⁴⁴
- **Seguridad en el CI/CD:** Asegurar la cadena de integración continua y despliegue continuo con controles de acceso adecuados y una segmentación que prevenga la inyección de código malicioso durante el proceso de compilación y despliegue.⁴²

3.9. A09:2021 - Fallos de Registro y Monitoreo de Seguridad

Esta categoría, que subió del puesto 10 en 2017 al 9 en 2021³, se refiere a la falta de registro, monitoreo y alertas de eventos de seguridad que son críticos para la detección de ataques en

curso y el análisis forense.¹

Ejemplos de Explotación

- **Brechas Indetectadas:** La falta de monitoreo de registros de acceso puede permitir que un ataque de exfiltración de datos se mantenga activo durante años sin ser detectado. Un ejemplo de la vida real es un caso en el que se informó a un proveedor de servicios de salud que un atacante había modificado miles de registros sensibles durante más de siete años debido a la ausencia de registros de seguridad.⁴⁶
- **Ahogamiento de Datos:** Un sistema que registra un volumen excesivo de eventos sin priorizar, como errores no críticos, puede generar una "fatiga de alerta" que lleva a los equipos de seguridad a ignorar notificaciones críticas.⁴⁷
- **Falta de Contexto en los Registros:** Los registros que no contienen el contexto adecuado (como el ID del usuario, la hora de la transacción o el ID de seguimiento) hacen que sea imposible rastrear la actividad de un atacante después de un incidente.⁴⁸

Mitigación

- **Registro de Eventos Críticos:** Registrar todos los fallos de inicio de sesión, cambios de privilegios, intentos de acceso no autorizado y transacciones de alto valor con suficiente contexto de usuario para permitir el análisis forense.⁴⁶
- **Monitoreo en Tiempo Real:** Implementar sistemas de gestión de eventos e información de seguridad (SIEM) para monitorear, correlacionar y alertar sobre actividades sospechosas en tiempo real o casi real.⁴⁵ Se debe asegurar que las alertas sean claras y priorizadas para evitar la sobrecarga de información.⁴⁷
- **Plan de Respuesta a Incidentes:** Establecer un plan formal de respuesta a incidentes (basado en marcos como el NIST 800-61r2) para actuar rápidamente cuando se detecta un ataque.⁴⁶

3.10. A10:2021 - Server-Side Request Forgery (SSRF)

SSRF es una vulnerabilidad que permite a un atacante engañar a un servidor para que realice una solicitud HTTP a un recurso interno o externo no autorizado, actuando como un proxy a las peticiones del atacante.¹

Ejemplos de Explotación

- **Escaneo de Red Interna:** Un atacante puede manipular la URL de entrada para que el servidor escanee puertos y servicios internos que no están expuestos a Internet, evadiendo los *firewalls* perimetrales.⁵⁰ Esto le permite identificar otros objetivos potenciales dentro de la red privada de la organización.⁵²

- **Acceso a Metadatos de la Nube:** Un atacante puede utilizar la vulnerabilidad SSRF para acceder a los metadatos de la instancia de un servicio en la nube.⁵³ Estos metadatos a menudo contienen credenciales o claves de API temporales que pueden ser utilizadas para acceder a otros recursos dentro del mismo entorno de la nube.⁵⁵
- **Omisión de Controles de Acceso:** Algunos servicios internos solo permiten el acceso desde la interfaz de *localhost*.⁵² Un atacante puede utilizar SSRF para forzar al servidor a realizar una solicitud a este servicio, eludiendo así los controles de seguridad y accediendo a paneles de administración o datos sensibles.⁵¹

Mitigación

- **Listas Blancas (*Whitelisting*):** La mitigación más sólida contra SSRF es implementar una lista blanca de los dominios y URLs a los que la aplicación tiene permitido conectarse.⁵⁰ A diferencia de las listas negras (que prohíben dominios específicos), las listas blancas son más difíciles de eludir.
- **Validación de Entradas Rigurosa:** Validar y sanear toda la entrada del usuario, especialmente en los parámetros de la URL, para asegurarse de que no contenga direcciones IP privadas o dominios internos.¹
- **Autenticación en Servicios Internos:** Siguiendo el principio de confianza cero, se debe habilitar la autenticación para todos los servicios internos, incluso si solo se supone que son accesibles desde dentro de la red. Esto asegura que, incluso si un atacante explota una vulnerabilidad SSRF, no podrá acceder a los recursos internos sin las credenciales adecuadas.⁵⁵
- **Deshabilitar Esquemas de URL no Utilizados:** Deshabilitar los protocolos de URL no utilizados como *file:///* o *gopher://* para prevenir ataques que exploten estos protocolos.⁵⁵

4. Una Visión Holística: La Interconexión de las Vulnerabilidades en Ataques en Cadena

Las vulnerabilidades de seguridad rara vez se explotan de forma aislada. Los atacantes combinan múltiples fallas en "cadenas de explotación" para aumentar la probabilidad de éxito y el impacto del ataque.¹⁵ Comprender estas interconexiones es crucial para construir una defensa en profundidad que no solo repare fallas individuales, sino que también contenga los ataques cuando una capa de seguridad se ve comprometida.

Los siguientes son ejemplos que ilustran cómo se encadenan las vulnerabilidades de OWASP:

- **Inyección + Control de Acceso:** Un atacante puede comenzar con una vulnerabilidad de inyección SQL (A03) para acceder a la base de datos de una aplicación. Al examinar los datos, podría encontrar identificadores de usuario o roles (A01) que le permitan

manipular su propia sesión para escalar privilegios de un usuario regular a un administrador.³⁸

- **Componentes Vulnerables + Mala Configuración de Seguridad:** La brecha de Equifax es un ejemplo clásico de esta cadena.³⁸ El ataque comenzó con una vulnerabilidad en el componente de *software* de código abierto Apache Struts (A06). Sin embargo, el fallo no habría llevado a una brecha masiva si no se hubiera encadenado con una mala configuración de seguridad (A05) de Equifax, que no había aplicado el parche de seguridad disponible a tiempo. Esta combinación de un componente obsoleto y una falta de gestión de parches creó el vector de ataque.³⁸
- **Fallos Criptográficos + Falta de Monitoreo:** Un ataque de "hombre en el medio" (*Man-in-the-Middle*) puede explotar una vulnerabilidad en un sitio web que no aplica el uso de HTTPS (A02), lo que le permite al atacante robar la *cookie* de sesión del usuario.⁷ Este ataque puede pasar desapercibido si la aplicación carece de un registro y monitoreo adecuados (A09) de los eventos de seguridad o si las alertas están mal configuradas. En este caso, la falta de visibilidad permite que el ataque avance sin ser detectado durante un período prolongado.

Vulnerabilidad de Entrada	Vulnerabilidad de Salida	Escenario de Encadenamiento
A03 Inyección	A01 Control de Acceso Roto	Un atacante usa SQLi para obtener los ID de usuario de una tabla y luego explota un fallo de IDOR para acceder a la información de otros usuarios.
A06 Componentes Vulnerables	A05 Mala Configuración de Seguridad	Un atacante explota una vulnerabilidad en un <i>framework</i> obsoleto para acceder a un directorio expuesto con credenciales por defecto.
A02 Fallos Criptográficos	A07 Fallos de Identificación	Un atacante usa el tráfico en claro (HTTP) para robar un token de sesión (A02) y secuestrar la identidad del

		usuario (A07).
A10 SSRF	A01 Control de Acceso Roto	Un atacante utiliza un servidor vulnerable a SSRF para acceder a un panel de administración interno que no requiere autenticación en el localhost.

5. Estrategia de Mitigación Integral: Un Plan Proactivo

La defensa de una aplicación moderna contra las vulnerabilidades de OWASP requiere una estrategia integral que va más allá de la simple corrección de errores de codificación. Un plan proactivo debe abarcar todo el ciclo de vida del desarrollo de software y construir múltiples capas de seguridad.

- **Seguridad desde el Diseño (Shift Left):** La inclusión de A04: Diseño Inseguro en la lista de 2021 destaca que las vulnerabilidades a menudo se originan en la fase de diseño.⁴ Las organizaciones deben integrar la seguridad en esta etapa a través del modelado de amenazas, la definición de requisitos de seguridad y la adopción de arquitecturas de referencia seguras.³
- **Adopción de un Modelo de Confianza Cero:** El principio de confianza cero, que asume que ninguna entidad (ni siquiera los usuarios o servicios internos) es inherentemente confiable, es una guía poderosa.⁵⁵ En lugar de depender de la seguridad del perímetro, se deben implementar controles de seguridad, como la MFA (A07) y la autenticación de servicios internos (A10), que verifican la identidad y el acceso en cada punto de interacción.¹⁷
- **Estrategia en Capas y Automatización:** Las mitigaciones deben aplicarse en múltiples capas para evitar el encadenamiento de ataques. El uso de herramientas automatizadas, como el análisis de seguridad de aplicaciones estático (SAST) para el código fuente y el análisis de composición de *software* (SCA) para las dependencias, ayuda a identificar problemas temprano en el SDLC.² El monitoreo y la respuesta a incidentes (A09) se convierten en la última capa de defensa, permitiendo a la organización detectar y contener un ataque que haya eludido las defensas anteriores.⁴⁵
- **Cultura de Seguridad:** La seguridad es una responsabilidad compartida.¹ La capacitación continua de los desarrolladores, auditores y equipos de operaciones es crucial para asegurar que las políticas y los procedimientos de seguridad se implementen y se sigan de manera consistente.

6. Conclusiones y Futuro de la Seguridad de Aplicaciones

El OWASP Top 10: 2021 refleja un panorama de amenazas que ha madurado, pasando de ataques oportunistas a cadenas de explotación estratégicas.¹⁵ La subida de vulnerabilidades relacionadas con el diseño (A04), la cadena de suministro (A08) y la visibilidad (A09) demuestra que la industria ha reconocido la necesidad de un enfoque más holístico y menos reactivo de la seguridad. La lucha contra estas vulnerabilidades ya no es una simple cuestión de parchear el código, sino de diseñar sistemas seguros desde sus cimientos y de monitorearlos continuamente para detectar comportamientos anómalos.

El futuro de la seguridad de las aplicaciones exige una integración profunda de la seguridad en cada etapa del desarrollo, una mentalidad de confianza cero y una cultura organizacional que priorice la protección de los activos digitales. El OWASP Top 10 sigue siendo una herramienta esencial para la concienciación y un recordatorio de que las amenazas más críticas de hoy a menudo tienen sus raíces en fallos fundamentales que se pueden y se deben prevenir.

7. Anexo: Tabla de Referencia Rápida de Mitigaciones

Vulnerabilidad	Riesgo Principal	Principales Controles de Mitigación
A01 Broken Access Control	Divulgación de datos y escalada de privilegios	Principio de Mínimo Privilegio (PoLP), validación del lado del servidor, RBAC.
A02 Cryptographic Failures	Exposición de datos sensibles	Cifrado robusto en tránsito y en reposo, <i>hashing</i> seguro de contraseñas.
A03 Injection	Ejecución de comandos no autorizados	Consultas parametrizadas, saneamiento y validación de entradas.

A04 Insecure Design	Fallas arquitectónicas	Modelado de amenazas, uso de patrones de diseño seguros, seguridad en el SDLC.
A05 Security Misconfiguration	Acceso no autorizado, divulgación de datos	Proceso de endurecimiento automatizado, eliminación de características innecesarias.
A06 Vulnerable and Outdated Components	Ejecución de código remoto, compromiso del sistema	Inventario continuo de componentes, gestión de parches, SCA.
A07 Identification and Authentication Failures	Secuestro de cuentas, relleno de credenciales	Autenticación multifactor (MFA), gestión de sesiones segura.
A08 Software and Data Integrity Failures	Ataques a la cadena de suministro, manipulación de datos	Firmas digitales para actualizaciones, repositorios de confianza, seguridad en CI/CD.
A09 Security Logging and Monitoring Failures	Fallos en la detección de ataques, análisis forense ineficaz	Registro de eventos críticos, monitoreo en tiempo real, plan de respuesta a incidentes.
A10 Server-Side Request Forgery (SSRF)	Escaneo de red interna, acceso a recursos privados	Listas blancas (<i>whitelisting</i>) de URLs, validación rigurosa de entradas.

Obras citadas

1. What Is the OWASP Top 10 Vulnerabilities? The List and Mitigation Methods - EC-Council, fecha de acceso: septiembre 15, 2025, <https://www.eccouncil.org/cybersecurity-exchange/penetration-testing/owasp-top-10-vulnerabilities-mitigation/>
2. What Is the OWASP Top 10 and How Does It Work? - Black Duck, fecha de acceso: septiembre 15, 2025, <https://www.blackduck.com/glossary/what-is-owasp-top-10.html>

3. OWASP Top Ten, fecha de acceso: septiembre 15, 2025, <https://owasp.org/www-project-top-ten/>
4. Introduction - OWASP Top 10:2021, fecha de acceso: septiembre 15, 2025, https://owasp.org/Top10/A00_2021_Introduction/
5. OWASP Top 10:2021, fecha de acceso: septiembre 15, 2025, <https://owasp.org/Top10/>
6. OWASP 2017 vs OWASP 2021 - MetaCompliance, fecha de acceso: septiembre 15, 2025, <https://support.metacompliance.com/hc/en-gb/articles/13886421827089-OWASP-2017-vs-OWASP-2021>
7. A02 Cryptographic Failures - OWASP Top 10:2021, fecha de acceso: septiembre 15, 2025, https://owasp.org/Top10/A02_2021-Cryptographic_Failures/
8. ¿Qué es el OWASP? ¿Qué es el OWASP Top 10? - Cloudflare, fecha de acceso: septiembre 15, 2025, <https://www.cloudflare.com/es-es/learning/security/threats/owasp-top-10/>
9. OWASP A04 Insecure Design: Risks, Examples, and Prevention - Vumetric, fecha de acceso: septiembre 15, 2025, <https://www.vumetric.com/blog/owasp-top-10-a04-insecure-design-everything-you-need-to-know/>
10. What is OWASP? OWASP Top 10 Vulnerabilities & Risks - F5, fecha de acceso: septiembre 15, 2025, <https://www.f5.com/glossary/owasp>
11. ¿Qué es OWASP? OWASP Top 10 de vulnerabilidades y riesgos - F5, fecha de acceso: septiembre 15, 2025, https://www.f5.com/es_es/glossary/owasp
12. Que es el Broken Access Control con ejemplos y codigos de casos especificos - 4Geeks, fecha de acceso: septiembre 15, 2025, <https://4geeks.com/es/lesson/que-es-el-broken-access-control-con-ejemplos-y-codigos-de-casos-especificos>
13. Broken Access Control | 5 ejemplos reales - Ransomware Help, fecha de acceso: septiembre 15, 2025, <https://www.ransomwarehelp.com/es/vectores-de-ataque/broken-access-control/>
14. Broken access control: The leading OWASP Top 10 security risk - Invicti, fecha de acceso: septiembre 15, 2025, <https://www.invicti.com/blog/web-security/broken-access-control/>
15. A01 Broken Access Control - OWASP Top 10:2021, fecha de acceso: septiembre 15, 2025, https://owasp.org/Top10/A01_2021-Broken_Access_Control/
16. How Best to Mitigate Broken Access Control: Top Practices - Dahlcore Security Guard Services, fecha de acceso: septiembre 15, 2025, <https://dahlcore.com/blog/f/how-best-to-mitigate-broken-access-control-top-practices>
17. What Is a Security Misconfiguration? - Fortinet, fecha de acceso: septiembre 15, 2025, <https://www.fortinet.com/resources/cyberglossary/security-misconfiguration>
18. What is Cryptographic Failure? Real-life Examples, Prevention, Mitigation - Certera, fecha de acceso: septiembre 15, 2025,

<https://certera.com/blog/what-is-cryptographic-failure-real-life-examples-prevention-mitigation/>

19. Cryptographic Failures Real-Life Examples - QAwerk, fecha de acceso: septiembre 15, 2025, <https://qawerk.com/blog/cryptographic-failure/>
20. Prevent OWASP A02 Cryptographic Failures: A Quick Guide - Vumetric, fecha de acceso: septiembre 15, 2025, <https://www.vumetric.com/blog/owasp-top-10-a02-cryptographic-failures/>
21. OWASP Top 10 - A03 Injection Vulnerabilities: Risks & Fixes - Vumetric, fecha de acceso: septiembre 15, 2025, <https://www.vumetric.com/blog/owasp-top-10-a03-injection-vulnerabilities-analyzed/>
22. Injection Flaws - OWASP Foundation, fecha de acceso: septiembre 15, 2025, https://owasp.org/www-community/Injection_Flaws
23. A03 Injection - OWASP Top 10:2021, fecha de acceso: septiembre 15, 2025, https://owasp.org/Top10/A03_2021-Injection/
24. Injections - The Many Faced Threat - Code Intelligence, fecha de acceso: septiembre 15, 2025, <https://www.code-intelligence.com/blog/types-of-injection-vulnerabilities>
25. 5 consejos para evitar ataques de inyección SQL - PowerDMARC, fecha de acceso: septiembre 15, 2025, <https://powerdmarc.com/es/sql-injection-prevention/>
26. Cómo prevenir un ataque de inyección SQL - Cloudflare, fecha de acceso: septiembre 15, 2025, <https://www.cloudflare.com/es-es/learning/security/threats/how-to-prevent-sql-injection/>
27. What is OWASP? What is the OWASP Top 10? | Cloudflare, fecha de acceso: septiembre 15, 2025, <https://www.cloudflare.com/learning/security/threats/owasp-top-10/>
28. A04 Insecure Design - OWASP Top 10:2021, fecha de acceso: septiembre 15, 2025, https://owasp.org/Top10/A04_2021-Insecure_Design/
29. Understanding Insecure Design: A Deep Dive - Hadrian.io, fecha de acceso: septiembre 15, 2025, <https://hadrian.io/blog/understanding-insecure-design-a-deep-dive>
30. Insecure Design — ThreatNG Security - External Attack Surface Management (EASM) - Digital Risk Protection, fecha de acceso: septiembre 15, 2025, <https://www.threatngsecurity.com/glossary/insecure-design>
31. Insecure Design | Tutorials & Examples - Snyk Learn, fecha de acceso: septiembre 15, 2025, <https://learn.snyk.io/lesson/insecure-design/>
32. OWASP A05 Risk: Fixing Security Misconfigurations Quickly - Vumetric, fecha de acceso: septiembre 15, 2025, <https://www.vumetric.com/blog/owasp-top-10-a05-security-misconfiguration-and-security-settings-examined/>
33. A05:2021 – Security misconfiguration - Zerolynx, fecha de acceso: septiembre 15, 2025, <https://www.zerolynx.com/en/blogs/news/securitymisconfiguration>
34. What is a Security Misconfiguration? Types & Examples - JumpCloud, fecha de

- acceso: septiembre 15, 2025,
<https://jumpcloud.com/blog/what-is-a-security-misconfiguration>
35. Security Misconfiguration: Impact, Examples, and Prevention, fecha de acceso: septiembre 15, 2025, <https://brightsec.com/blog/security-misconfiguration/>
36. A05 Security Misconfiguration - OWASP Top 10:2021, fecha de acceso: septiembre 15, 2025,
https://owasp.org/Top10/A05_2021-Security_Misconfiguration/
37. A06 Vulnerable and Outdated Components - OWASP Top 10:2021, fecha de acceso: septiembre 15, 2025,
https://owasp.org/Top10/A06_2021-Vulnerable_and_Outdated_Components/
38. Las 10 principales vulnerabilidades de OWASP y sus soluciones - Xygeni, fecha de acceso: septiembre 15, 2025,
<https://xygeni.io/es/blog/owasp-top-10-and-their-remedies/>
39. ¿Tu web está en riesgo? Descubre por qué las librerías obsoletas son una bomba de tiempo - Flu Project, fecha de acceso: septiembre 15, 2025,
<https://www.flu-project.com/2025/02/VulnerableandOutdatedComponents.html>
40. A07:2021 – Fallas de Identificación y Autenticación - Consultoría en Ciberseguridad, fecha de acceso: septiembre 15, 2025,
<https://blindspotcs.com/owasp-top-10/fallas-de-identificacion-y-autenticacion/>
41. A07 Identification and Authentication Failures - OWASP Top 10:2021, fecha de acceso: septiembre 15, 2025,
https://owasp.org/Top10/A07_2021-Identification_and_Authentication_Failures/
42. Software and data integrity failures: An OWASP Top 10 risk - Invicti, fecha de acceso: septiembre 15, 2025,
<https://www.invicti.com/blog/web-security/software-and-data-integrity-failures-a-n-owasp-top-10-risk/>
43. Fallas en el software y en la integridad de los datos: Explicación, ejemplos, prevención, fecha de acceso: septiembre 15, 2025,
<https://qawerk.es/blog/fallas-en-el-software-y-en-la-integridad-de-los-datos/>
44. A08 Software and Data Integrity Failures - OWASP Top 10:2021, fecha de acceso: septiembre 15, 2025,
https://owasp.org/Top10/A08_2021-Software_and_Data_Integrity_Failures/
45. Risk of Security and Monitoring Logging Failures | USA, fecha de acceso: septiembre 15, 2025,
<https://www.softwaresecured.com/post/risk-of-security-and-monitoring-logging-failures>
46. A09 Security Logging and Monitoring Failures - OWASP Top 10:2021, fecha de acceso: septiembre 15, 2025,
https://owasp.org/Top10/A09_2021-Security_Logging_and_Monitoring_Failures/
47. Los errores más comunes en la implementación de sistemas de monitorización y cómo evitarlos - Solucions-IM, fecha de acceso: septiembre 15, 2025,
<https://www.solucions-im.com/es/los-errores-mas-comunes-en-la-implementacion-de-sistemas-de-monitorizacion-y-como-evitarlos>
48. Buenas prácticas para logs : r/devsarg - Reddit, fecha de acceso: septiembre 15, 2025,

https://www.reddit.com/r/devsarg/comments/1gamqxi/buenas_pr%C3%A1cticas_para_logs/

49. Pasos Esenciales para Identificar y Mitigar Vulnerabilidades - Fiberlux, fecha de acceso: septiembre 15, 2025,
<https://fiberlux.pe/pasos-esenciales-para-identificar-y-mitigar-vulnerabilidades/>
50. OWASP A10 SSRF: Understanding Server Side Request Forgery, fecha de acceso: septiembre 15, 2025,
<https://www.vumetric.com/blog/owasp-top-10-a10-server-side-request-forgery-ssrf/>
51. A10:2021 – Server-Side Request Forgery (SSRF) - Zerolynx, fecha de acceso: septiembre 15, 2025, <https://www.zerolynx.com/blogs/news/ssrf>
52. What is SSRF (Server-side request forgery)? Tutorial & Examples | Web Security Academy, fecha de acceso: septiembre 15, 2025,
<https://portswigger.net/web-security/ssrf>
53. Server Side Request Forgery - A10 OWASP Top 10 - Wallarm, fecha de acceso: septiembre 15, 2025, <https://www.wallarm.com/what/server-side-request-forgery>
54. Cómo explotar la vulnerabilidad SSRF - YouTube, fecha de acceso: septiembre 15, 2025, <https://www.youtube.com/watch?v=wSii0GOhyWo>
55. 7 SSRF Mitigation Techniques You Must Know - Bright Security, fecha de acceso: septiembre 15, 2025,
<https://brightsec.com/blog/7-ssrf-mitigation-techniques-you-must-know/>
56. Mitigación de las 10 principales amenazas de seguridad de las API de OWASP en Azure API Management | Microsoft Learn, fecha de acceso: septiembre 15, 2025,
<https://learn.microsoft.com/es-es/azure/api-management/mitigate-owasp-api-threats>